

Term Project: Multiplication Tables

CP 431 – Parallel Programming

Winter 2025

Group 3

March 30, 2025

Instructor: I. Kotsireas, e-mail ikotsire@wlu.ca

Member 1 Name: Lukas Palubiski ID: 169037714

palu7714@mylaurier.ca

- Graphing, documentation

Member 2 Name: Amirhossein Ahmadi Noori ID: 169042860

ahma2860@mylaurier.ca

- Code creation, documentation

Member 3 Name: Bassil Ahmad ID: 200450090

ahma0090@mylaurier.ca

- Code creation & execution, documentation

Member 4 Name: Yuvraj Singh ID: 200670570

sing7057@mylaurier.ca

- Code creation and documentation

Member 5 Name: Ryan Soomal ID: 210370340

soom0340@mylaurier.ca

- Serial code creation, testing

Table of Contents

Table of Contents	3
Problem Introduction and Unique Properties:	4
Code Explanation:	4
Overview:	4
Workload Balancing/Distribution:	5
Example:	5
Algorithm Approach:	6
Merging Process:	8
Example:	8
Considerations:	9
Results:	10
Confirmed By-Hand Examples:	10
Example 1 (N = 5):	10
Example 2 (N = 10):	11
Final Solution Code Results:	12
Infrastructure Details:	13
Hardware:	14
Software and Build System:	14
Appendix A (Solution Code):	15
Appendix B (Solution Code Results):	18
Appendix C (Various Helper Scripts):	20
References:	22

Problem Introduction and Unique Properties:

The problem entails finding the number of different elements in an $N \times N$ multiplication table. Some key points to note are the symmetric nature of a $N \times N$ multiplication table where the elements on either side of the diagonal are the same. This allows us to more efficiently search for duplicate elements as we only need to iterate over $(n * (n+1))/2$ elements. Moreover, the longest row length is equal to N , with each subsequent row length being $N-1$. This pattern allows us to calculate the row and column values given the start index + iteration offset of a processor which when multiplied together returns the desired value. These unique properties allow us to develop an efficient parallel program to compute the number of different elements in an $N \times N$ multiplication table.

Code Explanation:

Overview:

The solution code is available under [Appendix A](#). The format of the code is as follows. To begin, we initialize, then distribute the workload among the processors and ensure that processors get similar amounts of work. Next, we process iterates over it designated workload which is the number of elements it must check. This is done via the `calcValueFromIdx()` function which calculates the current index of the element that needs to be retrieved from the matrix and calculates that value based on the found row and column. From there the result is inserted into a

local set ensuring only unique values persist in a sorted order, important for the merging process. After the process finishes loading all unique values into its set the merging process begins using a tree-based merging system. The result of the system is process rank 0 containing the final vector that holds all the unique values in the $N \times N$ multiplication matrix. Process 0 outputs this result along with the time taken using MPI time.

Workload Balancing/Distribution:

Given an $N \times N$ matrix, to calculate the total number of unique elements that we need to check, we only need to count half of the matrix in addition to the diagonal, since it is symmetric. Therefore, `globalElems` is calculated by the formula $(n * (n + 1)) / 2$. Each processor works on an equal number of elements with the maximum difference between any processor's workload being 1 element. This distribution is achieved by dividing `globalElems` by the number of processors and adding 1 if the rank is less than the remainder. To continue, the start index of each processor begins at the end index of the previous processor's workload, calculated and stored in `startIdx`.

Example:

Processors, $P = 6$

$N \times N$ matrix, $n = 100,000$

Total elements to go over, `globalElems` = 5,000,050,000

rank	numElems
0	833,341,667
1	833,341,667
2	833,341,667

3	833,341,667
4	833,341,666
5	833,341,666

Total = 5,000,050,000

rank	startIdx
0	0
1	833,341,667
2	1,666,683,334
3	2,500,025,001
4	3,333,366,668
5	4,166,708,334

Algorithm Approach:

The approach of our algorithm starts by only taking values in the upper triangle of the symmetric $N \times N$ multiplication table. Because we want equal load balancing among all processors it is important that the number of elements each processor processes is balanced, where the difference is no greater than 1 element. This is accomplished via the formula discussed above in the workload distribution section which allows processors to span multiple rows, incorporating all the elements they must evaluate and insert into the set data structure. Given the start element # of a processor (startIdx) and the offset (i in the for loop) along with the max row length (n in an $N \times N$ matrix), the calcValueFromIdx() function computes the value at that location. This is accomplished by checking if the position (startIdx) is within the current row's

element size ($n-1$ for each row). Once this is true, the col value is calculated and subsequently the value as it is the multiplication of the row and column values.

X	1	2	3	4	5	6	7	8	9	10
1	1	2	3	4	5	6	7	8	9	10
2		4	6	8	10	12	14	16	18	20
3			9	12	15	18	21	24	27	30
4				16	20	24	28	32	36	40
5					25	30	35	40	45	50
6						36	42	48	54	60
7							49	56	63	70
8								64	72	80
9									81	90
10										100

Image 1: Example of load balancing method

$P = 4, n = 10$

Process	numElems
0	14
1	14
2	14
3	13

Merging Process:

After each processor has computed the unique values present in the range they cover and added them to localSet, the merging process begins. The set is converted to a vector as MPI communication protocols such as MPI_Send and MPI_Recv do not accept the set data structure. The use of the set data structure ensures that the data is sorted, a property which is retained when the set is converted to a vector which will be useful later in the merging process.

The processors merge in a pairwise fashion in rounds starting with step = 1 and doubling step at each iteration. A process which fits the condition of $(\text{my_rank} \% (2 * \text{step}) == 0)$ has a rank which is a multiple of $2 * \text{step}$, this process receives data from a partner process with rank = $\text{my_rank} + \text{step}$. The transfer of data from the partner process to the receiving process is done in parts, first sending the size of the vector and then the vector. The receiving process uses the set_union function to merge its own sorted vector with the received vector, not including duplicates. The resulting vector contains the unique elements from the ranges the two processors (and previous processors if it isn't the first round) covered. The final iteration leaves process 0 with the resulting vector containing unique elements for the whole $(n * (n+1))/2$ range.

Example:

Process	localVec
0	2,6,11
1	4,5,6
2	2,8,10
3	4,10,11

Iteration 1:

- Step = 1

- **Process 0 receives** data ($0 \% 2 == 0$)
 - Result = {2,4,5,6,11}
- **Process 2 receives** data ($2 \% 2 == 0$)
 - Result = {2,4,8,10,11}
- **Processes 1 and 3 send** data

Iteration 2:

- Step = 2
- **Process 0 receives** data ($0 \% 4 == 0$)
 - Result = {2,4,5,6,8,10,11}
- **Process 2 sends** data

Note: This is an example scenario with made up data to illustrate the merging process.

Considerations:

- Merging is not fully efficient as it doesn't utilize all processors, partner processors send data and let the receiving processor perform the union calculations. Moreover, at each iteration less processors are involved in the merging process leading to downtime.
- The `std::set` data structure from the C++ standard library uses a tree implementation, which has a complexity of $O(\log n)$ for insertion. A potential test in the future would be to replace `std::set` with `std::unordered_set` which has a complexity (on average) of $O(1)$ for insertion. There are two alterations that could be made to implement this:
 - Sort the unordered sets before the merging process begins once on each processor which would be $O(n \log(n))$ complexity but only occur once.
 - Implement an alternative merging approach that doesn't require sorted ranges like `set_union` does.

Results:

Confirmed By-Hand Examples:

Example 1 (N = 5):

1. Verify by hand that that $M(5) = 14$

X	1	2	3	4	5
1	<u>1</u>	<u>2</u>	<u>3</u>	<u>4</u>	<u>5</u>
2	2	4	<u>6</u>	<u>8</u>	<u>10</u>
3	3	6	<u>9</u>	<u>12</u>	<u>15</u>
4	4	8	12	<u>16</u>	<u>20</u>
5	5	10	15	20	<u>25</u>

Image 2: 5x5 table. Upper triangular highlighted.

$M(N)$ = # of different elements in an $N \times N$ multiplication matrix.

Each element is defined as $A_{ij} = i \cdot j$, for $1 \leq i, j \leq N$

Here we have the multiplication table of 1 to 5, unique elements are underlined

Writing out all the unique elements we have:

$[1, 2, 3, 4, 5, 6, 8, 9, 10, 12, 15, 16, 20, 25] = 14$ elements

Therefore matrix (5) = 14

Example 2 (N = 10):

1. Verify by hand that that M (10) = 42

X	1	2	3	4	5	6	7	8	9	10
1	1	2	3	4	5	6	7	8	9	10
2	2	4	6	8	10	12	14	16	18	20
3	3	6	9	12	15	18	21	24	27	30
4	4	8	12	16	20	24	28	32	36	40
5	5	10	15	20	25	30	35	40	45	50
6	6	12	18	24	30	36	42	48	54	60
7	7	14	21	28	35	42	49	56	63	70
8	8	16	24	32	40	48	56	64	72	80
9	9	18	27	36	45	54	63	72	81	90
10	10	20	30	40	50	60	70	80	90	100

Image 3: 10x10 table.

Similarly, we find:

Unique elements

[1,2,3,4,5,6,7,8,9,10,12,14,15,16,18,20,21,24,25,27,28,30,32,35,36,40,42,45,48,49,50,54,56,60,63,64,70,72,80,81,90,100] = 42 elements.

Therefore $M(10) = 42$

Final Solution Code Results:

The following are the results of the solution code, using $P = 80$, and computing N for the values specified in the original problem. The code was run three times to get a decent average runtime (See [Appendix B](#) for the individual results of each run).

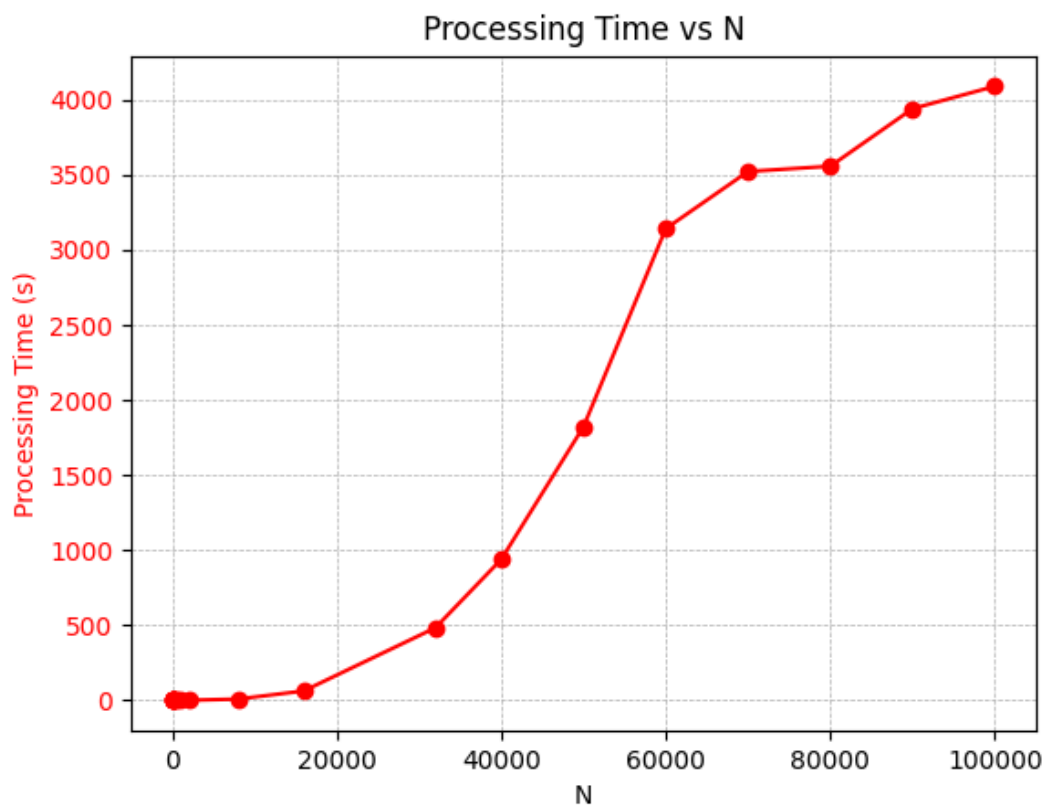


Image 4: Graph of processing time vs. value of N using the average times of the 3 runs

As N increases beyond 60,000, the program using 80 processors becomes more efficient because computation time grows relative to communication overhead. The workload is better balanced across processors, leading to a leveling off in processing time. At very large N, the parallel implementation has a higher efficiency, as processors spend more time computing rather than waiting for communication.

N Values	Resulting Unique Values in Matrix	Average Processing Time
5	14	0.01226935
10	42	0.02008355
20	152	0.01645795
40	517	0.0172571
80	1939	0.01811675
160	7174	0.02252015
320	27354	0.011346275
640	103966	0.02710415
1000	248083	0.05833585
2000	959759	0.1950655
8000	14509549	8.095155
16000	56705617	62.12955
32000	221824366	482.9255
40000	344462009	938.316
50000	534772334	1822.38
60000	766265795	3141.58
70000	1061907538	3522.45
80000	1742684297	3557.54
90000	2588769104	3940.93
100000	2832228693	4090.13

Average Results Table: Table of results from above graph.

Infrastructure Details:

Hardware:

All testing was done on the TEACH cluster (more info in [1]) using a maximum of 2 nodes, each having 40 processors and 188 GiB of RAM. The TEACH cluster has 8 nodes in total, as of the time of writing.

Software and Build System:

Our language of choice was C++, for the benefit of the standard library. We used *openmpi 5.0.3* as our MPI library.

The TEACH cluster runs Rocky Linux 9.5 and comes with a module loading mechanism which makes installation of any necessary libraries/modules very simple. The following four commands were run to setup openmpi:

```
module load gcc/13.3
```

```
module load intel/2024.2.0
```

```
module load nvhpc/25.1
```

```
module load openmpi/5.0.3
```

After running these commands, usage of commands like `mpic++` and `mpirun` is simple.

Unfortunately, these commands must be run in every session. This will be discussed later.

To build our program, the following command was used:

```
mpic++ o- progName ./progName.cpp
```

Since this command was relatively short, a script was not created for this.

To make things easier, we created some scripts to help with running the program. Scripts allowed us to run the program with varying parameters (such as number of processors and size of N) and pipe the output to files as needed. Importantly, it also allowed us to load the necessary modules in the script, freeing us from this task.

Scripts also allowed us to take advantage of SLURM, the TEACH clusters job scheduling system. This allowed us to tweak how many nodes we were able to use, allowing us to use up to 80 processors, instead of just 40.

Unfortunately, the TEACH cluster limits the maximum runtime of any single job to 4 hours. As a result, we had to split the program executions into separate jobs that could run in under 4 hours. In this case, we put all N s from 5-80k in one job, and 90k-100k in another. To get results in a timely manner, we submitted both jobs simultaneously, allowing the scheduler to begin execution as soon as resources were available. As an aside, for certain hours of the day during the last week of testing, our jobs were waiting in the queue for up to an hour, likely because they were using 2 nodes each.

[Appendix C](#) contains an example script we used to run our code. We had slight variations of this code to account for the 4-hour time limit, as mentioned above, but there were no other major modifications.

Appendix A (Solution Code):

```
#include <mpi.h>
```

```
#include <iostream>
#include <set>
#include <vector>
#include <algorithm>

// Set to store values
std::set<int> localSet;

int calcValueFromIdx(int idx, uint64_t chunkLen)
{
    int row = 0;
    int pos = idx;
    int curN = chunkLen;

    while (pos >= curN)
    {
        row++;
        pos -= curN;
        curN--;
    }

    int col = chunkLen - curN + pos;
    return (row + 1) * (col + 1);
}

int main(int argc, char **argv)
{
    int my_rank, p;
    double max_time;
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &my_rank);
    MPI_Comm_size(MPI_COMM_WORLD, &p);

    if (argc < 2)
    {
        if (my_rank == 0)
            std::cerr << "Usage: " << argv[0] << " <n>" << std::endl;
        MPI_Finalize();
        return 1;
    }

    double time = -MPI_Wtime();
```

```
uint64_t n = std::stoi(argv[1]);
uint64_t globalElems = n * (n + 1) / 2;

int baseElems = globalElems / p;
int remainder = globalElems % p;

int numElems = baseElems + (my_rank < remainder ? 1 : 0);
int startIdx = my_rank * baseElems + std::min(my_rank, remainder);

for (int i = 0; i < numElems; ++i)
{
    int val = calcValueFromIdx(startIdx + i, n);
    localSet.insert(val);
}

// Convert the set into a vector for MPI communication
std::vector<int> localVec(localSet.begin(), localSet.end());

// Tree-style merging: processes merge their vectors in rounds
int step = 1;
while (step < p) {
    if (my_rank % (2 * step) == 0) {
        int partner = my_rank + step;
        if (partner < p) {
            // Receive partner's vector and merge
            int partnerSize;
            MPI_Recv(&partnerSize, 1, MPI_INT, partner, my_rank+1, MPI_COMM_WORLD,
MPI_STATUS_IGNORE);
            std::vector<int> recvVec(partnerSize);
            MPI_Recv(recvVec.data(), partnerSize, MPI_INT, partner, my_rank+2,
MPI_COMM_WORLD, MPI_STATUS_IGNORE);

            // Create merged vector
            std::vector<int> mergedVec;
            mergedVec.reserve(localVec.size() + recvVec.size());

            // Since vectors are already sorted, use set_union to merge them and remove duplicates
            std::set_union(localVec.begin(), localVec.end(), recvVec.begin(), recvVec.end(),
std::back_inserter(mergedVec));

            // Update local vector
            localVec = std::move(mergedVec);
        }
    }
}
```

```
} else {  
    // Send local vector to the partner and exit  
    int partner = my_rank - step;  
    int localSize = localVec.size();  
  
    MPI_Send(&localSize, 1, MPI_INT, partner, partner+1, MPI_COMM_WORLD);  
    MPI_Send(localVec.data(), localSize, MPI_INT, partner, partner+2, MPI_COMM_WORLD);  
  
    break;  
}  
step *= 2;  
}  
  
time += MPI_Wtime();  
  
MPI_Reduce (&time, &max_time, 1, MPI_DOUBLE, MPI_MAX, 0, MPI_COMM_WORLD);  
  
// Print the number of unique values  
if (my_rank == 0)  
{  
    std::cout << "Number of unique values: " << localVec.size() << std::endl;  
    std::cout << "Time: " << max_time << " seconds" << std::endl;  
}  
  
MPI_Finalize();  
return 0;  
}
```

Appendix B (Solution Code Results):

The solution code was run 3 times. The results of each run are below:

N	Result	Processing Time (s)
5	14	0.0151429
10	42	0.00531653
20	152	0.0159152
40	517	0.00506677

80	1939	0.0201895
160	7174	0.00786929
320	27354	0.0278363
640	103966	0.0274711
1000	248083	0.0543783
2000	959759	0.192259
8000	14509549	8.11791
16000	56705617	62.1472
32000	221824366	482.411
40000	344462009	936.534
50000	534772334	1820.44
60000	766265795	3137.37
70000	1061907538	3570.34
80000	1742684297	3507.63
90000	2588769104	3971.52
100000	2832228693	4070.56

Results Table 1: First run

N	Result	Processing Time (s)
5	14	0.0074599
10	42	0.0239829
20	152	0.016982
40	517	0.0155199
80	1939	0.0173666
160	7174	0.019851
320	27354	0.0147216
640	103966	0.0267459
1000	248083	0.0609488
2000	959759	0.193514
8000	14509549	8.09489
16000	56705617	62.1861
32000	221824366	483.49
40000	344462009	939.645
50000	534772334	1825.23

60000	766265795	3145.88
70000	1061907538	3527.7
80000	1742684297	3565.36
90000	2588769104	3971.89
100000	2832228693	4063.2

Results Table 2: Second run

N	Result	Processing Time (s)
5	14	0.0170788
10	42	0.0161842
20	152	0.0159339
40	517	0.0189943
80	1939	0.0188669
160	7174	0.0251893
320	27354	0.00797095
640	103966	0.0274624
1000	248083	0.0557229
2000	959759	0.196617
8000	14509549	8.09542
16000	56705617	62.073
32000	221824366	482.361
40000	344462009	936.987
50000	534772334	1819.53
60000	766265795	3137.28
70000	1061907538	3517.2
80000	1742684297	3549.72
90000	2588769104	3909.97
100000	2832228693	4117.06

Results Table 3: Third run

Appendix C (Various Helper Scripts):

Script 1: Run program with all necessary values of N and pipe output to CSV file:

```
#!/bin/bash
#SBATCH --nodes=2
```

```
#SBATCH --ntasks-per-node=40
#SBATCH --job-name=MultTables
#SBATCH --output=output.txt
#SBATCH --time=04:00:00

#Usage: sbatch --export=C="/.tp",P=80 run_csv_iters.sh

#Load necessary modules
module load gcc/13.3
module load intel/2024.2.0
module load nvhpc/25.1
module load openmpi/5.0.3

output_file="output-P${P}.csv"
echo "N,Processing Time" > "$output_file"

#Loop over Ns
for N in 5 10 20 40 80 160 320 640 1000 2000 8000 16000 32000 40000 50000 60000 70000 80000
90000 100000;
do echo -n "$N," >> "$output_file"
  mpirun -np $P $C $N >> "$output_file"
done
```

Script 2: Graphing the Timing of each N value:

```
import matplotlib.pyplot as plt
import numpy as np

#data
N = [5, 10, 20, 40, 80, 160, 320, 640, 1000, 2000, 8000, 16000, 32000, 40000, 50000, 60000, 70000,
80000, 90000, 100000]
Processing_Time = [0.01226935, 0.02008355, 0.01645795, 0.0172571, 0.01811675, 0.02252015,
0.011346275, 0.02710415, 0.05833585,
0.1950655, 8.095155, 62.12955, 482.9255, 938.316, 1822.38, 3141.58, 3522.45, 3557.54,
3940.93, 4090.13]

#create plot
fig, ax2 = plt.subplots()
ax2.plot(N, Processing_Time, 'ro-', label='Processing Time')
ax2.set_xlabel('N')
ax2.set_ylabel('Processing Time (s)', color='r')
```

```
ax2.tick_params(axis='y', labelcolor='r')  
ax2.grid(True, linestyle="--", linewidth=0.5)  
plt.title('Processing Time vs N')  
plt.show()
```

References:

[1] SciNet. 2024. Teach — SciNet Documentation. Retrieved March 27, 2025 from <https://docs.scinet.utoronto.ca/index.php/Teach>